

Trace Replay for LLM Agents: Prefix Reuse and Semantic Boundaries

Halley Young

<https://github.com/thehalleyyoung/stepback>

Abstract—Debugging an LLM agent usually means asking a counterfactual question about a recorded run: what would the agent have done if one tool had returned something else, or if one call had used a different model? Re-running the agent answers the question at the price of every model call, and the re-run can drift from the recording before it reaches the point of interest. We present *stepback*, a recorder and replay engine that stores agent runs as signed traces of content-addressed steps and answers a substitution by re-executing only the steps whose declared inputs depend on it (the *dirty set*), with an agent adapter rebuilding state for live continuation. Dependencies are bound through a cumulative history digest, which is the correct dependency for agents whose prompts accumulate the conversation. On 1,200 public trajectories from τ -bench, SWE-agent and OpenHands, the dirty set of a single substitution is exactly the suffix after the substituted step in all 5,943 cases; a tool-output substitution at a random position avoids 48–52% of LLM calls and 33–40% of LLM tokens relative to full re-execution, and a substitution in the last tool observation avoids 86–96% of calls. In 47 live counterfactual runs on τ -bench retail, *stepback* reused the same 406 recorded LLM calls as LangGraph time travel and vcrpy cassette replay, while full re-runs reproduced the recorded prefix in only 36 of 47 tasks. Historical classification measurements are 1–3 ms. Fresh deterministic controls show that dependency hashes do not reconstruct changed message content and callback identity does not detect changed captured state. Versioned caches match an ordinary semantic-version cache on the controlled inputs; no incremental benefit over matched checkpoint or cache baselines is established.

I. INTRODUCTION

LLM agents interleave model calls, tool calls and, in conversational settings, replies from a user. When a run goes wrong, the trace of that run is the primary evidence, and the debugging questions are counterfactual: would the agent have recovered if the order lookup had failed, would a different model have issued the refund, which earlier tool output caused the bad action? Three practical approaches exist. Re-running the agent with a fault injected re-executes every model call, including the ones before the fault, and because provider outputs vary across runs even at temperature zero [1], the re-run may never reach the state the developer wanted to perturb. HTTP cassette tools such as vcrpy [2] serve byte-identical requests from a recording, which requires the agent to rebuild those requests exactly. Checkpointing frameworks such as LangGraph [3] can fork a run from a saved state, which requires the agent to be written against the framework and to have run with a checkpoint.

stepback treats a recorded run as a dependency graph of steps, each identified by the hash of its canonical inputs.

A substitution changes the inputs or outputs of some steps; *stepback* computes which steps are affected, serves every other step from the recording, and re-executes only the affected steps. The idea is the one behind incremental computation [4], [5] and build systems [6], applied to agent traces. The contribution of this paper is a tool that makes that idea work on real agent runs, and a measurement of what it buys.

Claim: Under complete declared history dependencies, static classification marks the recorded suffix of a substitution. This identifies potential re-execution work, rather than automatically reconstructing the counterfactual program state. The measured live prefix reuse ties conventional checkpoint and cassette baselines. Portable signed traces provide an inspectable artifact; this study does not establish improved task success or incident diagnosis.

Contributions:

- A trace format and replay engine in which steps bind their dependencies through a cumulative history digest, so early cutoff never reuses a step whose prompt contains a changed earlier output (Section III).
- An open-source implementation with a recorder that tracks parent steps correctly when frameworks run steps in copied execution contexts (Section IV).
- A static study on 1,200 public trajectories from three agent families and a live study on 50 τ -bench tasks comparing *stepback* with full re-runs, vcrpy and LangGraph time travel (Section V). Data, scripts and raw outputs are in the repository.

II. BACKGROUND AND RELATED WORK

Incremental computation and build systems: Self-adjusting computation [4] and Adapton [5] record a dependency graph during a first run and re-execute only the parts whose inputs changed. Build systems [6] do the same at file or task granularity and name two refinements *stepback* also uses: verifying traces (a task is rebuilt if the hash of any input differs from the recorded one) and early cutoff (if a rebuilt task reproduces its old output, its dependents need not be rebuilt). *stepback* applies verifying traces to agent steps. The new constraint is that the “program” is an LLM agent whose prompt is the whole conversation so far, which makes the history, not the immediate predecessor, the unit of dependency.

Record and replay: Deterministic record and replay for processes, as in rr [7], and omniscient or time-travel debugging [8] replay a recorded execution exactly. They do

not answer counterfactuals: once an input changes, replay diverges and the tool must hand control back to live execution. AgentRR [9] records agent interactions to derive reusable experience for future tasks; it does not compute which steps of one run a substitution affects. HTTP cassettes (vcrpy) replay at the request level using configured matchers, including custom normalization. The measured baseline matches request bodies.

Agent tooling: Observability platforms [10] record and display agent runs. Semantic caches such as GPTCache [11] serve similar prompts from a cache, trading exactness for hit rate. LangGraph checkpointers store graph state after each node and support forking from a past checkpoint with modified state. The `stepback` trace format and classifier operate independently of a framework checkpoint. Live continuation still requires the agent implementation and its state adapter. The dirty set is defined over recorded step dependencies.

Minimization: Delta debugging [12] shrinks a failure-inducing input. `stepback` includes `ddmin` over sets of substitutions; each probe is a replay, so its cost is the dirty set of the probe.

III. APPROACH

Traces: A trace is a sequence of steps $s_1, \dots, s_N$ in topological order. Each step has a kind (LLM call, tool call, router, parallel open/join, exception), canonical JSON inputs $I(s)$, outputs $O(s)$, and a parent $p(s)$. We write $h(x)$ for SHA-256 over canonical JSON. Frames on disk are chained with HMAC-SHA 256 and signed with Ed 25519, so a trace that has been edited after recording fails verification.

History binding: For each step the recorder computes a digest

$$D(s) = h([D(p(s)), h(O(s))]),$$

with $D(p(s))$ replaced by a null value when s has no parent or was recorded as independent of earlier steps. $D(s)$ covers the outputs of s and of every ancestor on its parent chain. A step recorded with history binding (the default for LLM and tool calls) stores $D(p(s))$ in a `context` field of $I(s)$. Steps may instead bind only $h(O(p(s)))$ (parent binding) or selected fields of the parent output.

The choice matters because replay applies early cutoff. Consider a ReAct agent [13] whose trace is $\ell_1, t_1, \ell_2, t_2, \ell_3$, with LLM calls ℓ_i and tool calls t_i . Substitute $O(t_1)$. With parent binding, ℓ_2 is dirty and is re-executed; if it reproduces its recorded output, t_2 and ℓ_3 see unchanged parent hashes and are reused, although the prompt of ℓ_3 contains the substituted tool output. With history binding, $I(\ell_3)$ contains $D(t_2)$, which depends on $O(t_1)$, so ℓ_3 is dirty regardless of what ℓ_2 returns. Parent binding remains available for steps that read nothing but their parent.

Dirty set: Given substitutions σ , the classifier walks the trace once. For step s it rebinds the `context` field from the current (possibly substituted) digests or hashes of its dependencies, applies the substitutions that target s , and marks s dirty if (1) a substitution forces its output, (2) $h(I'(s)) \neq h(I(s))$, (3) its parent’s output changed and s binds its parent,

or (4) its recorded nondeterminism class requires re-execution (unseeded randomness, wall clock). Without an executor, a dirty step’s new output is unknown and is represented by a fresh sentinel hash, which makes every dependent dirty. The classifier is a single pass, linear in the size of the trace.

Replay: Replay serves clean steps from the trace with zero executor calls and calls a user-supplied executor (LLM, tool, router) for dirty ones, consulting an optional content-addressed step cache keyed by (executor scope, kind, $h(I'(s))$). By default the scope is private to one executor; an explicit version namespace permits shared reuse under stable callback semantics and captured external state. Cache contents are trusted. When re-execution must reflect new content (an LLM call whose prompt should now include the substituted tool output), the agent itself has to continue: `stepback` supplies the first dirty step and the clean prefix, and an agent-specific adapter rebuilds the agent state from that prefix and resumes the agent live. For the τ -bench agent in Section V the adapter is about 50 lines.

Branches: Parallel fan-out is recorded as open and join steps; the join depends on every branch tail. A substitution inside one branch dirties that branch, the join and its dependents, and leaves sibling branches clean.

IV. IMPLEMENTATION

`stepback` is a Python package (reference implementation) with Rust crates for trace parsing and verification, read and verify bindings for TypeScript, Go, JVM and .NET, and a WASM verifier. The recorder exposes `llm_call`, `tool_call`, `router`, `parallel` and `exception`; provider shims wrap OpenAI, Anthropic, Bedrock, Gemini and others. Importers read OpenAI chat logs, LangSmith, OpenInference/OTel spans and other formats. The CLI covers recording, replay with substitutions, diffs, bisection, `ddmin` minimization, corpus sweeps and attestation packs.

Two implementation details affect correctness. First, history binding is rebound in every replay path: sequential replay, branch-parallel replay (digests travel with the per-region hash maps across tiers), the streaming event replay, the replay planner and the distributed classifier. Second, the recorder tracks the current parent per async task only inside async tasks and otherwise uses a recorder-wide parent. Frameworks such as LangGraph run each node in a worker thread with a copied `contextvars` context; a context-variable parent read inside such a copy is the value at copy time, which would attach every step to the first one.

V. EVALUATION

- RQ1 On real agent trajectories, how large is the dirty set of common substitutions, and how much LLM work does dirty-set replay avoid relative to full re-execution?
- RQ2 In live counterfactual runs, how does `stepback` compare with full re-runs, vcrpy cassette replay and LangGraph time travel in reused calls, cost and

TABLE I

SHARE OF LLM CALLS AND LLM TOKENS THAT DIRTY-SET REPLAY AVOIDS RELATIVE TO FULL RE-EXECUTION (POOLED OVER EPISODES).

Substitution	τ -bench		SWE-agent		OpenHands	
	calls	tokens	calls	tokens	calls	tokens
tool output, random	48.0%	34.9%	49.3%	33.2%	51.6%	39.6%
tool output, first	15.8%	6.8%	3.6%	0.7%	5.2%	0.7%
tool output, last	86.3%	82.2%	96.4%	95.4%	94.8%	93.4%
model swap, random	46.8%	35.0%	54.6%	41.0%	46.7%	35.1%
system prompt	0%	0%	0%	0%	0%	0%

time, and how often does each method preserve the recorded prefix?

RQ3 How long does dirty-set classification take?

A. RQ1 and RQ3: public trajectories

Setup: We imported 1,200 trajectories into signed stepback traces: 600 from the τ -bench historical trajectories [14] (the first 150 of each of the GPT-4o and Claude 3.5 Sonnet files for the retail and airline domains), 300 SWE-agent [15] trajectories from the Nebius dataset [16] (first shard, seeded sample), and 300 OpenHands [17] trajectories from the SWE-Gym release [18]. Each assistant message becomes an LLM step; each later tool result, environment observation or simulated-user turn becomes a tool step. All steps use history binding. Token counts use the o 200 k_base tokenizer on the full cumulative prompt of each call. Per episode we apply five substitutions: a tool output at a random tool step, at the first and at the last tool step that a later LLM call reads (simulated-user turns excluded), a model swap at a random LLM step, and a system-prompt change. Full re-execution of the episode is the baseline. Mean episode sizes are 25.9 steps (12.9 LLM calls, 41 k tokens) for τ -bench, 54.5 steps (27.8 calls, 321 k tokens) for SWE-agent and 37.7 steps (19.4 calls, 263 k tokens) for OpenHands.

Results: In all 5,943 (episode, substitution) cases the dirty set equals the suffix of the trace starting at the substituted step. These agents carry the whole conversation in every prompt, so every later step depends on every earlier output, and the classifier finds no step after the substitution point that could be reused. Table I gives the savings. Savings in tokens are lower than savings in calls because later calls have longer prompts: a random tool-output substitution avoids about half of the calls but a third of the tokens. Late substitutions, the common case when debugging the last action of a failed run, avoid 86–96% of calls, a $7.3\times$ (τ -bench), $27.8\times$ (SWE-agent) and $19.4\times$ (OpenHands) reduction in LLM calls. A system-prompt change dirties the whole trace. The median classification time per trace (RQ3) is 1.3–1.4 ms for τ -bench, 2.8–3.2 ms for SWE-agent and 2.0–2.8 ms for OpenHands on an Apple M-series laptop.

B. RQ2: live counterfactual replay

Setup: We implemented the τ -bench tool-calling agent (first tool call per turn, temperature 0) as a LangGraph graph with an agent node and an environment node, using

TABLE II

LIVE COUNTERFACTUAL REPLAY ON 47 τ -BENCH RETAIL TASKS.

Method	reused calls	live calls	cost (USD)	median time (s)	prefix kept
full re-run	0	1415	0.396	31.5	36/47
vcrrpy	406	961	0.305	25.1	47/47
LangGraph fork	406	971	0.296	24.3	47/47
stepback	406	952	0.277	23.9	47/47

the τ -bench retail tools, database, policy and user-simulator prompt, with gpt-4o-mini through OpenRouter for both agent and simulated user (seed 0). For each of the first 50 retail test tasks, one live run records a stepback trace, a vcrrpy cassette and LangGraph checkpoints simultaneously. The counterfactual replaces the output of the middle read-only tool call (a get_*, find_*, list_* or calculate call) with a service-unavailable error; 47 tasks have such a call. Four methods answer it, all running the same graph code for whatever must be executed live: *full* re-runs the episode live and injects the error at the same read-only call index; *vcrrpy* does the same under the recorded cassette (*new_episodes* mode, matching on method, URL and body); *LangGraph* updates the checkpoint after the target call and resumes from it; *stepback* classifies the trace, rebuilds the state at the first dirty step from the clean prefix and resumes. The environment database for a resumed run is rebuilt by re-applying the recorded tool actions locally.

Results: Table II reports totals over the 47 tasks. stepback, LangGraph and vcrrpy reuse exactly the same 406 recorded LLM calls (8.6 per task), and in every task the stepback dirty set is the trace suffix from the substituted call. The live calls after the substitution differ between methods because the continuation is sampled anew each time; their totals vary by 2% across the three prefix-reusing methods and say nothing about the methods themselves. Against full re-runs, prefix reuse cuts live calls by about a third and cost by 23–30%. The more consequential difference is in the prefix: in 11 of 47 full re-runs the agent took a different sequence of actions before the target call, so the injected error landed in a different state from the recorded one. Replay from the trace keeps the recorded prefix by construction. vcrrpy kept it as well because this agent rebuilds its requests byte for byte; volatile timestamps or identifiers can reduce hits under the measured body matcher, but custom normalization can address those differences. LangGraph uses saved graph checkpoints; stepback classification uses the trace, and its live continuation also uses the agent code, reconstruction adapter and environment. The historical API log, including the pilot, totals 1.7009994 USD; provider billing was not independently reconciled in this audit.

C. Fresh deterministic semantic controls

These local synthetic callbacks test replay semantics, not agent task quality. A tool originally returns A and a later model callback echoes its recorded message A. Substituting

tool output B invalidates both steps, but generic replay still passes message A; direct reconstruction passes B and returns B. Thus hash invalidation alone does not supply counterfactual message content.

A second frozen comparison warms twelve substituted integer inputs with a callback capturing offset 10, then changes that same callback’s captured offset to 20. Private instance scopes and a deliberately stale explicit namespace return stale outputs in all 12 cases with zero new calls. Explicit namespaces including the new offset and an ordinary memoizer keyed by name, arguments and semantic state version both match uncached direct execution in 12/12 cases, with 12 calls each. Stable warm repeats make zero new calls. Private scopes prevent unintended cross-instance sharing; they do not detect mutable closure state, remote model revisions or other hidden changes. The conventional baseline shows this is a familiar versioning obligation, not a novel caching advantage. The first run incorrectly compared wrapped trace outputs to raw callback outputs; that harness failure is retained and the corrected run compares callback values. All inputs and both runs are archived.

VI. DISCUSSION AND LIMITATIONS

On the agents we studied the dirty set of a single substitution is a suffix, so the reuse `stepback` achieves is the reuse of a checkpoint fork. Finer dirty sets come from structure that these agents do not have: independent parallel branches, independent work items processed by one worker (recorded as separate chains), and steps that read only their parent, where early cutoff applies. The repository’s tests exercise these cases on synthetic traces; we did not find them in the public trajectories.

Threats to validity: The live study covers one agent design, one domain, one model and one substitution type; other agents may rebuild state differently and the adapter is agent specific. The static study classifies substitutions without executing them, so it measures the work replay avoids, not the outcomes of the counterfactual runs; token counts use one tokenizer for all models. Tool side effects and other state outside the trace are assumed to be either captured as steps or rebuilt by the adapter; a hidden dependency outside the recorded graph is not detected. Semantic replay requires the recorder to capture every dependency, the adapter to reconstruct current message and tool-argument content, and shared caches to use stable executor versions and trusted entries. Hash equality alone cannot establish these conditions.

Evidence and verification scope: Public and live tables report historical measurements, independently recounted from saved records; the public classifier and paid continuations were not rerun. The formal model’s clean-output statement assumes recorder/full-replay coherence and does not establish a Python, adapter or cache refinement. Earlier full-suite failures and their logs are retained. Repaired focused checks passed; no final complete all-green suite is claimed. A held-out handoff or incident-diagnosis comparison against an attested checkpoint baseline remains necessary for a consequential tool claim.

VII. CONCLUSION

`stepback` supplies portable signed traces and dependency-based replay planning. On the studied cumulative-prompt agents, prefix reuse equals checkpoint and cassette reuse: all three methods save 406 recorded calls on 47 live tasks. Fresh controls locate the semantic boundary at state reconstruction and explicit executor-state versioning. This artifact supports reproducible inspection of that boundary; systematic efficiency or debugging-outcome advantages over the matched conventional baselines remain unestablished. The code and raw records are available at <https://github.com/thehalleyyoung/stepback>.

REFERENCES

- [1] B. Atil, S. Aykent, A. Chittams, L. Fu, R. J. Passonneau *et al.*, “Non-determinism of “deterministic” LLM settings,” 2024.
- [2] vcrpy contributors, “VCR.py: Automatically mock your HTTP interactions,” <https://github.com/kevin1024/vcrpy>, 2026, version 8.3.0.
- [3] LangChain, Inc., “LangGraph,” <https://github.com/langchain-ai/langgraph>, 2026, version 1.2.14, checkpoint `langgraph-checkpoint` 4.2.0.
- [4] U. A. Acar, G. E. Blelloch, and R. Harper, “Adaptive functional programming,” in *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2002, pp. 247–259.
- [5] M. A. Hammer, K. Y. Phang, M. Hicks, and J. S. Foster, “Adapton: Composable, demand-driven incremental computation,” in *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2014, pp. 156–166.
- [6] A. Mokhov, N. Mitchell, and S. Peyton Jones, “Build systems à la carte,” *Proceedings of the ACM on Programming Languages*, vol. 2, no. ICFP, pp. 79:1–79:29, 2018.
- [7] R. O’Callahan, C. Jones, N. Froyd, K. Huey, A. Noll, and N. Partush, “Engineering record and replay for deployability: Extended technical report,” 2017, short version in USENIX ATC 2017.
- [8] B. Lewis, “Debugging backwards in time,” 2003.
- [9] E. Feng, W. Zhou, Z. Liu, L. Chen, Y. Dong *et al.*, “Get experience from practice: LLM agents with record & replay,” 2025.
- [10] L. Dong, Q. Lu, and L. Zhu, “AgentOps: Enabling observability of LLM agents,” 2024.
- [11] F. Bang, “GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings,” in *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*, 2023, pp. 212–218.
- [12] A. Zeller and R. Hildebrandt, “Simplifying and isolating failure-inducing input,” *IEEE Transactions on Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002.
- [13] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” 2022.
- [14] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, “ τ -bench: A benchmark for tool-agent-user interaction in real-world domains,” 2024.
- [15] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “SWE-agent: Agent-computer interfaces enable automated software engineering,” 2024.
- [16] Nebius, “SWE-agent trajectories (dataset),” <https://huggingface.co/datasets/nebius/SWE-agent-trajectories>, 2024, revision 68195a14.
- [17] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang *et al.*, “OpenHands: An open platform for AI software developers as generalist agents,” 2024.
- [18] J. Pan, X. Wang, G. Neubig, N. Jaitly, H. Ji *et al.*, “Training software engineering agents and verifiers with SWE-Gym,” 2024.